



endingo

정훈이의 공부일지

CATEGORY



👋 안녕하세요, AI 개발자 이정훈입니다

AI · 백엔드 · 데이터 분석 중심의 프로젝트와 경험을 공유합니다.

 Contact Me

Data science/딥러닝

[딥러닝] 5. 다중 클래스 분류 모델링 - Iris

endingo 2025. 4. 7. 19:50

- ★ 다중클래스 분류의 목적함수는 `CrossEntropyLoss()`
- ★ 다중클래스 분류의 활성화함수는 `Softmax()` 지정할 필요없음
- ★ 은닉층의 활성화함수는 `ReLU()`
- ★ 다중클래스의 가변수화는 `integer incoding`

1. 데이터 준비

1) 데이터 준비

`data.drop(target, axis=1)`: target은 없애고 특성수만 존재하도록

`data.loc[:,target]`: target만 존재하도록

```
target = 'Species'
```

```
x = data.drop(target, axis = 1)
```

```
y = data.loc[:, target]
```



정훈이의 공부일지



- 다중 클래스 분류 모델링을 위해서는 y는 integer encoding을 먼저 수행해야됨
- LabelEncoder()는 문자열로 된 클래스를 정수로 변환해주는 클래스
- fit_transform(y)는 y 데이터에 있는 고유 클래스들을 학습하고, 각각을 0부터 시작하는 정수로 바꿔준다.

예를 들어, ['setosa', 'versicolor', 'virginica']가 있다면,

- setosa → 0
- versicolor → 1
- virginica → 2

```
le = LabelEncoder()
y = le.fit_transform(y)
y[:5]
```

```
array([0, 0, 0, 0, 0])
```

앞의 5개만 출력했다 => 모두 setosa였네?

- le.classes: 어떤 클래스들이 어떤 숫자로 매핑되었는지 확인 0부터 시작이다.

```
le.classes_
```

- le.inverse_transform(y): 다시 문자열로 되돌리고 싶으면

```
le.inverse_transform(y)
```

3) 데이터 분할

```
train_test_split()
훈련용: x_train, y_train
검증용: x_val, y_val
```

```
x_train, x_val, y_train, y_val = train_test_split(x, y, test_size=.3, random_state = 20)
```

4) 스케일링

```
범위값을 같게 조정
scaler.fit_transform
```

```
scaler = MinMaxScaler()
```



정훈이의 공부일지



2. 모델링

1) 딥러닝을 위한 준비작업

어레이 => 텐서 => 텐서 조합결합 => 데이터 로더

```
train_loader, x_val_ts, y_val_ts = make_DataSet(x_train, x_val, y_train, y_val, 32)
```

- 잘 전달됐는지 확인

첫번째 배치만 로딩해서 살펴보기

```
for x, y in train_loader:
    print(f"Shape of x [rows, columns]: {x.shape}")
    print(f"Shape of y: {y.shape} {y.dtype}")
    break
```

```
Shape of x [rows, columns]: torch.Size([32, 4])
Shape of y: torch.Size([32]) torch.int64
```

배치: 32

특성수: 4

2) 모델선언

다중 클래스 분류 같은 경우는 목적함수에 활성화함수가 포함되어있기에 신경망만 선언해주면됨

```
n_feature = x.shape[1]
# 외우기
n_class = len(le.classes_)
```

모델 구조 설계

```
model = nn.Sequential(
    nn.Linear(n_feature, n_class), # 출력층 활성화 함수를 지정하지 않음
).to(device)
```

```
print(model)
```

```
Sequential(
  (0): Linear(in_features=4, out_features=3, bias=True)
)
```

특성수 4로 들어와서 3개의 다중 클래스로 분류되는 신경망

3) 목적함수, 옵티마이저 선언



정훈이의 공부일지



```
loss_fn = nn.CrossEntropyLoss()    # Cross Entropy : 이 손실 함수는 내부에 SoftMax 연산이 포함(출력층 활성화 함수 지정 안함)
```

```
optimizer = Adam(model.parameters(), lr=0.1)
```

4) 모델 훈련

```
epochs = 100
```

```
tr_loss_list, val_loss_list = [], []
```

```
for t in range(epochs):
```

```
    tr_loss = train(train_loader, model, loss_fn, optimizer, device)
```

```
    val_loss, _ = evaluate(x_val_ts, y_val_ts, model, loss_fn, device)
```

```
    # 리스트에 loss 추가 --> learning curve 그리기 위해.
```

```
    tr_loss_list.append(tr_loss)
```

```
    val_loss_list.append(val_loss)
```

```
print(f"Epoch {t+1}, train loss : {tr_loss:4f}, val loss : {val_loss:4f}")
```

- 학습된 파라미터 확인

```
for name, param in model.named_parameters():
```

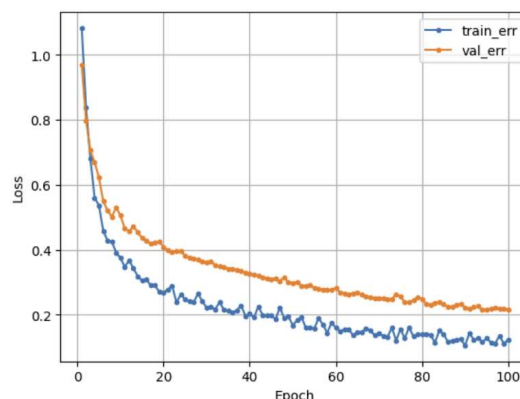
```
    if param.requires_grad:
```

```
        print(f"Parameter: {name}, Value: {param.data}")
```

```
Parameter: 0.weight, Value: tensor([[ -6.3295,  5.7793, -7.9037, -7.1722],
        [ 0.5912, -3.5310,  0.9848, -4.0391],
        [ 1.5848, -3.4844,  4.8391,  8.0727]])
Parameter: 0.bias, Value: tensor([ 5.8888,  4.3287, -6.7225])
```

- 학습 곡선

```
dl_learning_curve(tr_loss_list, val_loss_list)
```





정훈이의 공부일지



i. 예측결과를 일단 확인

evaluate()

1. 예측결과

_, pred = evaluate(x_val_ts, y_val_ts, model, loss_fn, device)

다중 분류의 예측 결과는 2차원 구조

pred.numpy()[:5]

```
array([[ 7.5821395,  2.495453, -7.709713 ],
       [-3.5028572,  2.2051692, -0.3795414],
       [-3.5675793,  1.8736591, -0.3805971],
       [-7.6794586,  1.6071429,  2.0758286],
       [-1.7996845,  2.538067, -1.535037 ]], dtype=float32)
```

ii. 결과값을 SoftMAX로 변환

nn.functional.softmax()

2. softmax로 변환

pred = nn.functional.softmax(pred, dim=1)

pred[:5]

```
tensor([[0.5723, 0.2149, 0.2127],
        [0.2187, 0.5460, 0.2353],
        [0.2209, 0.5348, 0.2443],
        [0.2323, 0.3565, 0.4112],
        [0.2177, 0.5631, 0.2192]])
```

확률값으로 변환되었다.

iii. 가장 큰 값의 인덱스를 표시하여 클래스 분류를 하는 것

3. 가장 큰 값의 인덱스

pred = np.argmax(pred.numpy(), axis = 1)

pred[:5]

```
array([0, 1, 1, 2, 1])
```

첫번째 행에서 가장 확률이 높은 것 0 세토사

두번째 행에서 가장 확률이 높은 것 1 버지칼라

세번째 행에서 가장 확률이 높은 것 1 버지칼라

네번째 행에서 가장 확률이 높은 것 2 버지니카

마지막 행에서 가장 확률이 높은 것 1 버지칼라

6) 혼동행렬에서 정보뽑기



정훈이의 공부일지



```
[ 0, 18,  0],
[ 0,  4, 10]])
```

```
print(classification_report(y_val_ts.numpy(), pred, target_names=le.classes_))
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	13
versicolor	0.82	1.00	0.90	18
virginica	1.00	0.71	0.83	14
accuracy			0.91	45
macro avg	0.94	0.90	0.91	45
weighted avg	0.93	0.91	0.91	45

3. 은닉층 추가

1) 모델선언

다중 클래스 분류에서 은닉층은 ReLU()로 사용하고

마지막 활성화함수는 Cross 목적함수가 softmax 함수를 품고있으므로 선언 안 해도된다.

```
n_feature = x.shape[1]
n_class = len(le.classes_)
```

모델 구조 설계

```
model = nn.Sequential(
    nn.Linear(n_feature, 4),
    nn.ReLU(),
    nn.Linear(4, 3),
    nn.ReLU(),
    nn.Linear(3, n_class),
).to(device)
```

```
print(model)
```

```
Sequential(
  (0): Linear(in_features=4, out_features=4, bias=True)
  (1): ReLU()
  (2): Linear(in_features=4, out_features=3, bias=True)
  (3): ReLU()
  (4): Linear(in_features=3, out_features=3, bias=True)
)
```

모델 평가일 때 각 각의 클래스의 예측값 구하기 => 확률화 => 확률 가장 높은 애 선택

공감

구독하기



정훈이의 공부일지



[블로그] > <블로그> >

[블로그] > <블로그> >

[딤퍼닝] 1. 딤퍼닝 개요 (0)

정훈이의 공부일지

개발하면서 겪는 시행착오와 배움을 기록합니다. 백엔드, AI, 데이터 분석, 프로젝트 경험 중심으로 운영 중입니다. 공부하면서 얻은 인사이트를 함께 나누고 싶어요!

구독하기

댓글 0

이름

비밀번호

로그인 댓글만 허용한 블로그입니다

등록

공지사항

최근에 올라온 글

최근에 달린 댓글

Total

- [딤퍼닝] 6. 다중 클래스 분류 모...
- [딤퍼닝] 5. 다중 클래스 분류 모...
- [딤퍼닝] 4. 이진분류 모델링
- [딤퍼닝] 3. 회귀 모델링

188

Today 9
Yesterday 114

링크

TAG

[more](#)

« 2025/04 »

글 보관함

- github
- git

일	월	화	수	목	금	토
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

2025/04 (16)
2025/03 (7)



정훈이의 공부일지



이정훈 개발자

AI와 백엔드, 데이터 분석을 좋아하는 개발자입니다.

다양한 문제 해결과 창의적인 개발을 추구합니다.



이메일 보내기